11/4/2026

# Fine-Grained Dog Breed Classification: Comparing Transfer Learning Against a Network Trained From Scratch

ITRI626 – Mini Project


**Group members**

JP Retief 38279118

Lindani Mdlalose 46290303

Sulaiman Sulaman 45427429

# Table of contents

Table of contents ..... 1
1. Abstract ..... 3
2. Introduction and problem motivation ..... 3
3. Dataset, classes, preparation and ethics ..... 4
3.1 Dataset ..... 4
3.2 Class definitions ..... 4
3.3 Composition, balance and image properties ..... 5
3.4 Cleaning ..... 7
3.5 Label checking ..... 8
3.6 Split and leakage ..... 9
3.7 Simplified condition ..... 9
3.8 Ethics ..... 9
4. Methodology, models and transfer learning ..... 10
4.1 Model Selection ..... 10
4.2 Custom CNN ..... 10
4.3 EfficientNetV2-S ..... 10
4.4 Swin-T ..... 11
4.5 Transfer learning strategy ..... 11
5. Experimental setup and hyperparameters ..... 11
5.1 Preprocessing and augmentation ..... 11
5.2 Training settings ..... 11
5.3 Fairness of the comparison ..... 12
5.4 Software, hardware and reproducibility ..... 12
5.5 Custom CNN development ..... 13
6. Results, figures and model comparison ..... 13
6.1 Overall Performance ..... 13
6.2 Learning Curve ..... 14
6.3 Per-Class performance ..... 15
6.4 Confusion matrices ..... 15
6.5 Example predictions ..... 17
7. Discussion, limitations and error analysis ..... 17

7.1 Why the models differ ..... 17
7.2 Error analysis ..... 18
7.3 What the comparison supports ..... 18
7.4 Limitations ..... 18
8. Conclusion ..... 19
9. References ..... 20
Appendix A: Per-class results ..... 21

# 1. Abstract

Four convolutional and transformer models were trained to classify photographs of dogs into 120 breeds using the Stanford Dogs dataset: a convolutional network built and trained from scratch, and three models initialised from ImageNet pretrained weights (MobileNetV3-Large, EfficientNetV2-S and Swin-T). All models used an identical 70/15/15 split, identical preprocessing and, for the three pretrained models, an identical two-phase training schedule.

The pretrained models reached 67.5 to 88 per cent test accuracy; the from-scratch network reached 53.84 per cent. Duplicate photographs within the dataset account for 0.2 percentage points or less across four independently trained models. A size-matched pretrained model, MobileNetV3-Large at 4,355,752 parameters against the baseline's 4,581,784, isolates pretraining at 13.69 points, leaving 20.50 points to capacity and architecture together. Transfer learning is therefore worth roughly two fifths of the difference rather than all of it.

The dataset itself was found to carry contradictory labels: 50 pairs of duplicate photographs are filed under two different breeds, affecting 40 of the 120 breeds, which places a ceiling on attainable accuracy. Seven of the twenty most frequent errors made by the from-scratch model fall on exactly those pairs.

# 2. Introduction and problem motivation

Fine-grained image classification assigns an image to one of several classes that all belong to the same broader category. It is difficult because the differences between classes are small and confined to certain parts of the image, while photographs of the same class vary widely in pose, lighting, scale and background. Dog breeds show this clearly: the Eskimo dog and the Siberian husky share coat pattern and build, and Stanford Dogs was constructed as a benchmark for exactly this kind of distinction (Khosla et al., 2011). This problem matters in life too. Breed identification is important for animal shelters, vets and pet owners. It’s often done by eye, which leads to mistakes and inaccurate records.

Deep learning models usually need a lot of labeled data to learn features, and fine-grained datasets often don’t have enough. The Stanford Dogs dataset for example has about 170 images per class. To get around this, researchers use transfer learning. That means starting with a model that was trained on a dataset like ImageNet (Deng et al., 2009) and then adjusting it for the new task. There’s a problem here. The Stanford Dogs dataset was taken from ImageNet so the pretrained models might have seen some of the images during training. That makes it hard to tell if the model’s success comes from learning or just from having seen those photos before.

This project aims to determine which of three model families, a custom convolutional neural network, the EfficientNetV2 family and the Swin Transformer family, is best suited for classifying dog breeds from photographs, as well as recommending a model on that basis. A secondary aim is to estimate how much of the pretrained models’ advantage can be given to pretraining, rather than the network size or to duplicate photographs within the dataset. This gets examined by including MobileNetV3-Large, a pretrained model with a similar number of parameters to the baseline, and by scoring the test set both with and without photographs that have near-duplicates in the training

data. The target classes are the 120 breeds defined in Stanford Dogs, each photograph is assigned to exactly one class, and all models are trained and evaluated on the same 70/15/15 split.

The report proceeds from the data to the models: Section 3 documents the dataset and its preparation, Section 4 describes the model families and the transfer learning strategy, and Section 5 sets out the experimental configuration. The results are presented in Section 6 and interpreted in Section 7, which also addresses classification errors and limitations, before Section 8 concludes with the recommended model.

# 3. Dataset, classes, preparation and ethics

## 3.1 Dataset

Stanford Dogs (Khosla *et al.*, 2011) was selected as the dataset for this project and comprises 20,580 photographs of 120 dog breeds. In addition to a breed label, every photograph is supplied with an annotation file marking the position of each dog in the image with a rectangular bounding box. The copy used in this project was obtained from Kaggle, at https://www.kaggle.com/datasets/jessicali9530/stanford-dogs-dataset.

The photographs and their breed labels were drawn from ImageNet (Deng *et al.*, 2009) rather than gathered specifically for the dataset (Khosla *et al.*, 2011). As a result, the dataset shares its source with the data used to pretrain three of the four models in this project, which affects how their test results are interpreted (Section 6.3). The applicable terms are therefore taken to be those of ImageNet, which restrict use of the database to non-commercial research and educational purposes (ImageNet, n.d.).

## 3.2 Class definitions

Each of the 120 classes corresponds to a single dog breed, defined by an ImageNet synset, which is a WordNet concept identified by a code such as n02085620 for the Chihuahua. The breed definitions are therefore inherited from ImageNet's taxonomy and were not redefined for this project. Figure 1 shows a representative photograph from each of the 120 classes.

The full set of 120 breeds was used as the primary classification task. Retaining the complete set keeps the task consistent with how the dataset is normally used and avoids making the problem easier by excluding the breeds that are hardest to separate. The effect of merging the most similar breeds is instead examined separately, through the simplified condition described in Section 3.7.

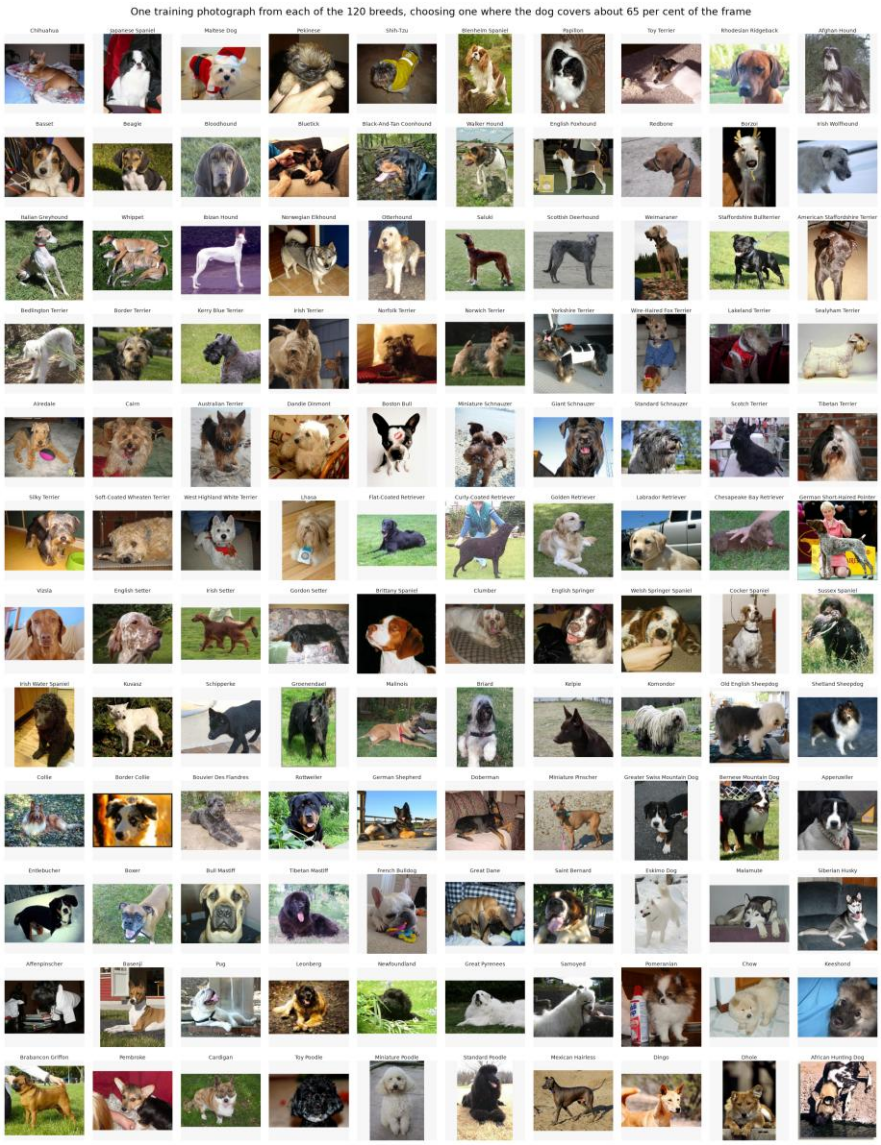



*Figure 1: Representative photographs from each of the 120 Stanford Dogs classes*

## 3.3 Composition, balance and image properties

*Table 1: Dataset composition and image properties*

| **Measure** | **Value** |
|---|---|
| Photographs | 20,580 |
| Breeds (classes) | 120 |
| Photographs per breed | 148 (redbone) to 252 (Maltese dog) |
| Mean, median per breed | 171.5, 159.5 |
| Largest to smallest class ratio | 1.70 to 1 |
| Median image size | 500 × 375 pixels |

| Measure | Value |
|---|---|
| Smaller than 224 pixels on one side | 1,577 (7.66%) |
| Median share of frame occupied by the dog | 55.3% |
| Photographs with more than one dog | 6.60% |

As shown in Table 1 the number of photographs for each breed varies from 148 for the redbone to 252 for the dog. The average is 171.5. The median is 159.5. The distribution is right-skewed as seen in Figure 2. Most breeds have 150 photographs while a few have much more. The ratio of the class to the smallest class is 1.70 to 1, which is considered a minor imbalance. Despite this the macro-averaged F1 score is reported along with accuracy. This is because -averaged F1 treats every breed equally so it does not get inflated by good results, in the larger classes.

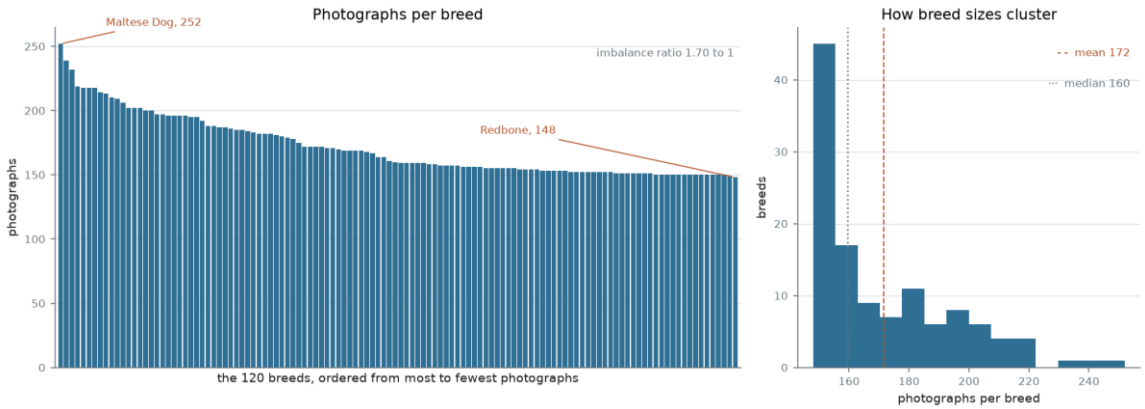



*Figure 1: Photographs per breed, sorted, and the distribution of breed sizes*

Image dimensions vary widely, from 97 to 3,264 pixels in width and from 100 to 2,562 pixels in height, with median dimensions of 500 by 375 pixels. The distribution shows a sharp boundary at 500 pixels on both axes, suggesting most photographs were downscaled to a maximum of 500 pixels on their longest side before publication, reducing the fine detail available to distinguish breeds. In addition, 7.66 per cent of photographs (1,577) fall below 224 pixels on at least one side, the input size used by all models, and are enlarged at load time with some loss of sharpness. Most photographs are landscape (68.72 per cent), with 25.95 per cent portrait and 5.33 per cent approximately square, so resizing to a square input alters the proportions of most images (Section 5.1).

The bounding-box annotations show that the dog occupies a median of 55.3 per cent of the frame and that in 14.52 per cent of photographs it occupies less than a quarter. Furthermore, 6.60 per cent of photographs contain than one dog, up to six, in a single image while carrying only one breed label. Such photographs are inherently ambiguous since a model may correctly identify a dog in the image without matching the labelled one an effect examined in Section 7.2.

## 3.4 Cleaning

Cleaning began with an integrity check of every file in the dataset. All 20,580 photographs and their 20,580 annotation files were present, and every photograph was decoded without error. The only formatting problem found was a single photograph in RGBA format, which includes a transparency channel, and it is converted to RGB at load time so that all inputs share the same three colour channels.

Duplicate photographs were searched for next, because the specification requires that near-identical images are not divided between training and test data. Byte-for-byte copies were found by comparing a unique digital fingerprint (a SHA-256 hash) of every file, yielding 89 groups of identical files, of which 37 fell into different partitions. Because resized or recompressed copies of the same photograph have different bytes, two perceptual fingerprints were also computed for every image, a difference hash (dHash) and a perceptual hash (pHash), each of which changes only slightly when an image is altered slightly. Choosing the matching threshold required judgement, so seven thresholds were evaluated: agreement between the two methods fell from 93.9 per cent at the strictest setting to 13.4 per cent at a distance of 10, and visual inspection of sampled pairs showed that matches beyond a distance of 6 were frequently different photographs. A pair was therefore treated as duplicate only if both methods agreed, with a difference-hash distance of 5 or less and a perceptual-hash distance of 10 or less, which identified 173 confirmed pairs, 74 of them split across partitions.

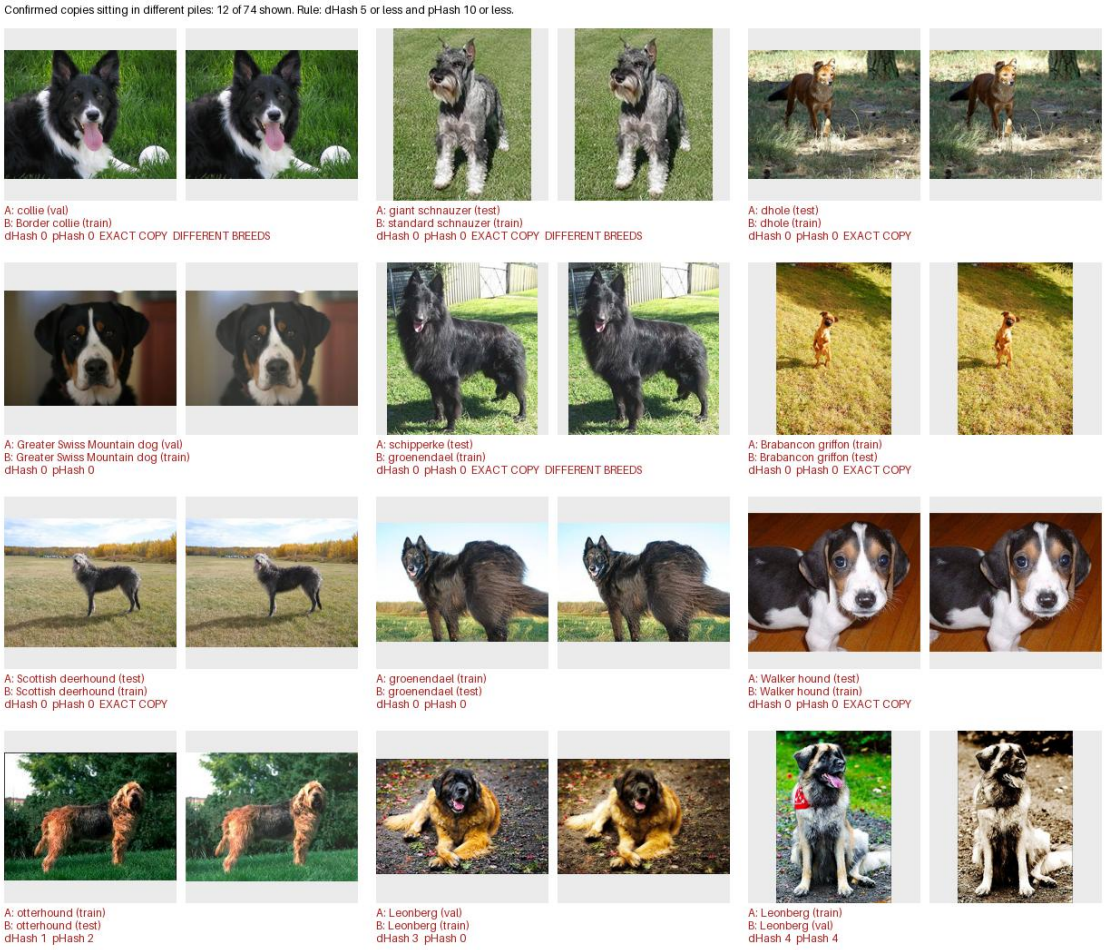


*Figure 3: Confirmed duplicate pairs that fall in different partitions. A red caption marks a photograph in the test set whose copy is in the training set.*

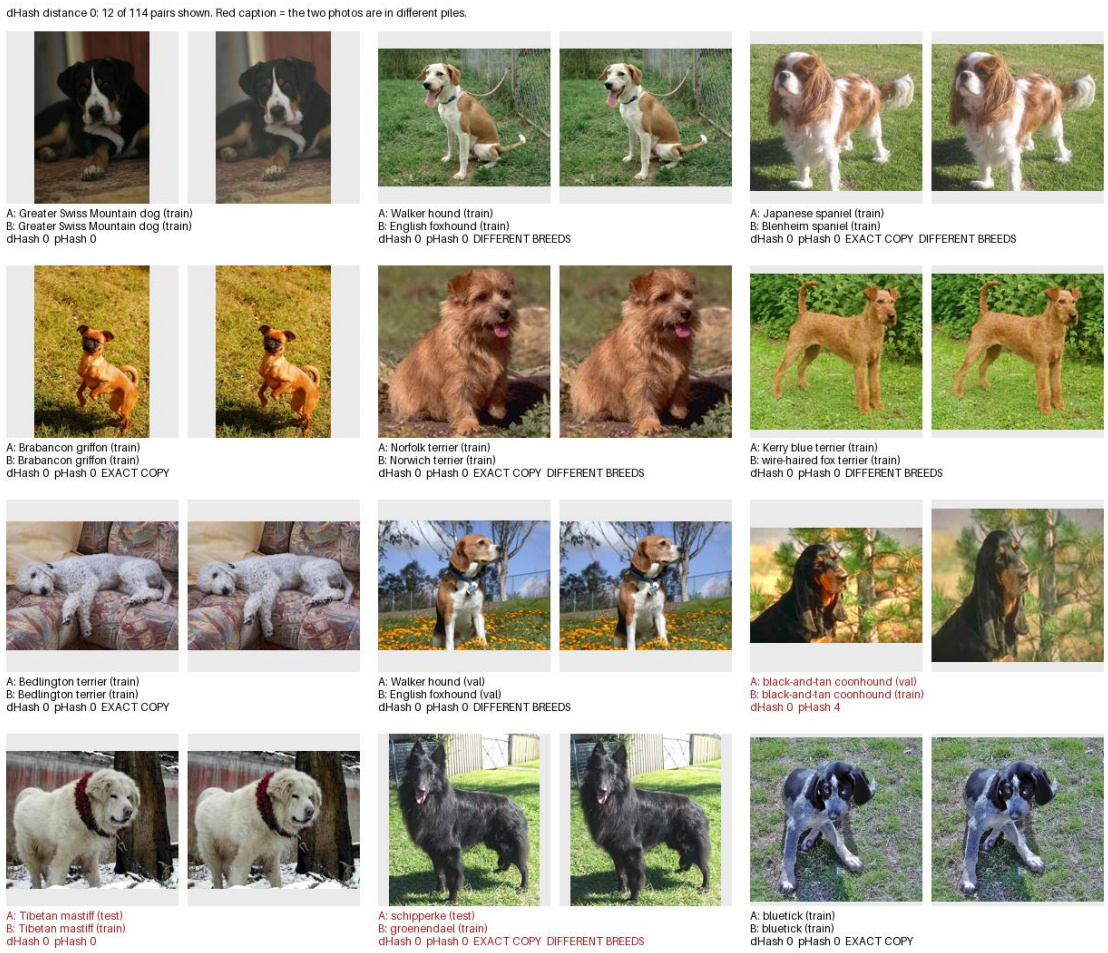



*Figure 4: Pairs at a difference-hash distance of zero. Several carry two different breed labels, which is how the contradictory labels were found.*

In the main 120-class condition, no photographs were removed: the duplicates were instead flagged and their effect on test performance measured (Section 3.6), so that all models could continue to use the shared split. Removal was applied only in the simplified condition, in which 170 duplicate and leaked photographs were excluded (Section 3.7).

## 3.5 Label checking

Labels in Stanford Dogs are those assigned when the photographs were collected for ImageNet, and no relabeling is reported by the dataset's authors (Khosla et al., 2011). ImageNet's labels were verified by crowd workers through Amazon Mechanical Turk rather than by breed experts (Deng et al., 2009), so their reliability for closely related breeds could not be assumed. Among the duplicate pairs identified in Section 3.4, 50 carry two different breed labels, and these pairs involve 40 of the 120 breeds. In most such cases, one of the labels must be wrong, although a photograph containing more than one dog could legitimately appear under two breeds.

## 3.6 Split and leakage

The data was split 70/15/15 into training, validation and test sets, stratified by breed so that each breed appears in the same proportion in every set, using random seed 42. The sets are slightly uneven because per-breed counts were rounded down, with the remainder added to the test set. The split was checked before use: it can be regenerated exactly from the seed, no photograph appears in more than one set, every label matches its breed folder, and all 120 breeds appear in every set.

The split was made by random selection and did not account for duplicates, so 34 test photographs (1.06 per cent) have a near-duplicate in the training set. Instead of regenerating the split, the test set was scored twice, with and without these 34 photographs, so that the effect of leakage could be measured (Section 6.5). This measures leakage rather than preventing it, which is listed as a limitation in Section 7.4.

## 3.7 Simplified condition

The simplified condition merges four groups of near-identical breeds and removes 170 duplicate and leaked photographs, leaving 114 classes (Table 2). Its purpose is to show how much accuracy is lost to breeds that the dataset labels inconsistently. Results on 114 classes are not directly comparable with those on 120, because merging removes the hardest decisions, so the two are always reported separately.

*Table 1: The two data conditions*

| Condition | Classes | Training | Validation | Test |
|---|---|---|---|---|
| Original (120 class) | 120 | 14,355 | 3,025 | 3,200 |
| Simplified (114 class) | 114 | 14,257 | 2,989 | 3,164 |
| Difference | 4 groups merged | 98 removed | 36 removed | 36 removed |

## 3.8 Ethics

Many photographs include people who could be identified, and neither source records their consent. This project collected no personal data, and the models only predict dog breeds. The trained models are used only in a private demonstration app and are not publicly deployed or shared.

# 4. Methodology, models and transfer learning

## 4.1 Model Selection

*Table 3: The model families compared*

| Model | Family | Pretrained weights | Parameters | Trainable in phase 2 | Role |
|---|---|---|---|---|---|
| Custom CNN | VGG-style CNN | None | 4,581,784 | All (from scratch) | Baseline, no pretraining |
| EfficientNetV2-S | EfficientNet | ImageNet-1K | 20,331,208 | 74.0% | Efficient CNN |
| Swin-T | Swin Transformer | ImageNet-1K V1 | 27,611,634 | About 56% | Transformer |
| MobileNetV3-Large | MobileNet | ImageNet-1K V1 | 4,355,752 | 72.0% | Size-matched control |

Three model families were compared (Table 3): a custom convolutional neural network (CNN) trained from scratch, EfficientNetV2-S and Swin-T. They were chosen to contrast learning from scratch with transfer learning, and convolution with attention. MobileNetV3-Large (Howard et al., 2019) is added as a control: it is pretrained but about the same size as the custom CNN, so comparing the two separates the effect of pretraining from the effect of size.

## 4.2 Custom CNN

The custom CNN follows a simple VGG-style design (Simonyan & Zisserman, 2015): five blocks of 3×3 convolution, batch normalisation, ReLU and max pooling, with the number of channels doubling from 32 to 512 as the image shrinks from 224 to 7 pixels. A second convolution in the last two blocks brings the network to 4,581,784 parameters, and global average pooling, dropout and a single output layer complete it. Global average pooling, which averages each feature map into one value, was used instead of a large fully connected layer, saving about 524,000 parameters.

## 4.3 EfficientNetV2-S

EfficientNetV2-S is the smallest model in the EfficientNetV2 family (Tan & Le, 2021), a convolutional design built for both accuracy and training speed. Its main ideas are scaling the network's depth, width and resolution together, and using simpler, faster "fused" blocks in the early layers. With 20.3 million parameters, it was chosen as a strong but practical convolutional network that can run on ordinary hardware.

## 4.4 Swin-T

Swin-T (Liu *et al.*, 2021) is a vision transformer, a model that uses self-attention to compare parts of an image with each other directly, rather than scanning it with small filters as a CNN does. A standard Vision Transformer (Dosovitskiy *et al.*, 2021) becomes expensive on larger images and needs very large training sets, which makes it a poor fit for about 20,000 photographs. Swin computes attention within small, shifting windows instead of across the whole image, and merges patches as it goes deeper, much as a CNN shrinks its feature maps. Swin-T is the smallest standard version, with 27.6 million parameters.

## 4.5 Transfer learning strategy

Transfer learning begins with a network that has already been trained on ImageNet. The final layer is replaced with one that is designed for 120 breeds. The training process happens in two parts. The first part has 2 epochs with the pretrained layers locked. The second part has 3 epochs with the layers unlocked but at a smaller learning rate. Locking the layers first stops random updates from the new layer from damaging the features that were learned from ImageNet. Then the lower learning rate helps those features adjust to dog breeds without being changed.

The amount of the network that was unlocked was matched by parameters not by layers. This was 72 to 74 per cent for the two CNNs. The structure of Swin-T is not as detailed so unlocking its stage leaves about 56 per cent of the parameters that can be changed. This difference is mentioned of being kept secret. All models use the ImageNet-1K weights. This was chosen on purpose so that no model starts with a trained version, than the others..

# 5. Experimental setup and hyperparameters

## 5.1 Preprocessing and augmentation

All photographs are resized directly to 224×224 pixels, normalised with the ImageNet mean and standard deviation, which the pretrained models expect. Resizing without cropping makes non-square photographs look distorted. It keeps the whole dog in the frame. A center crop could cut part of the dog off. Training images are changed by adding flips, rotating up to 15 degrees and adding a little colour jitter. Validation images and test images remain unchanged. These changes mimic photographs. A dog facing left is the breed, as a dog facing right. Photographs are often a little tilted and lighting can vary. Strong colour changes were avoided because a dog's coat colour helps people identify the breed.

## 5.2 Training settings

*Table 2: Training Configuration*

| **Setting** | **Pretrained models** | **Custom CNN** |
|---|---|---|
| Optimiser | AdamW, weight decay 0.01 | AdamW, weight decay 1e-4 |
| Learning rate | 3e-4 (phase 1), 1e-5 (phase 2) | 1e-3, cosine decay |

| Setting | Pretrained models | Custom CNN |
|---|---|---|
| Epochs | 2 + 3 | Up to 60, early stopping (patience 5) |
| Loss | Cross-entropy, label smoothing 0.1 | Cross-entropy |
| Batch size | 32 | 64 |
| Mixed precision, seed | Yes, 42 | Yes, 42 |

Table 4 lists the training settings, which are identical for the three pretrained models and differ for the custom CNN only where training from scratch requires it. Training from scratch needs far longer, so the custom CNN ran for up to 60 epochs with early stopping and a higher learning rate.

## 5.3 Fairness of the comparison

To keep the comparison fair, all models share the same split, input size, augmentation, normalisation and evaluation metrics. EfficientNetV2-S and MobileNetV3-Large share identical training and evaluation code, differing only in the model definition. The custom CNN's longer schedule gives it more opportunity to learn, not less, so the differences do not favour the pretrained models.

## 5.4 Software, hardware and reproducibility

*Table 3: Hardware and training time*

| Model | GPU | Training time |
|---|---|---|
| Custom CNN | Tesla T4 (Kaggle), 14.6 GB | 64.7 min (59 epochs) |
| EfficientNetV2-S | RTX 4050 Laptop, 8 GB | 14.7 min |
| Swin-T | RTX 4060 Laptop, 8 GB | 9.1 min |
| MobileNetV3-Large | RTX 4050 Laptop, 8 GB | 13.87 min |

All models were implemented in PyTorch 2.11.0 (Paszke *et al.*, 2019) with torchvision 0.26.0 and CUDA 12.8, on Windows 11 and Kaggle's Linux environment. As different GPUs were used, the times indicate scale only, and the clearer contrast is 5 epochs for the pretrained models against 59 for the custom CNN. Seeding fixes the data order and starting weights, but not every GPU operation; repeated runs differed by up to 1.2 points of macro F1, which is taken as the noise level for all comparisons. All development decisions were made on the validation set, and the test set was scored once per final model. Every run saves its settings, library versions and GPU to a configuration file, and full reproduction steps are given in the README.

## 5.5 Custom CNN development

*Table 6: Custom CNN development runs*

| Run | Change | Val macro F1 | Train−val gap |
|---|---|---|---|
| 1 | Initial design | 0.4969 | 15.8 |
| 2A | Stronger regularisation | 0.4911 | 8.9 |
| **2B** | **Extra convolution in blocks 4 and 5** | **0.5649** | **33.0** |
| 3A | 2B + stronger regularisation | 0.5602 | 27.2 |
| 3B | 2B over 120 epochs | 0.5270 | 23.9 |

The custom CNN's final configuration was chosen from five runs, each changing one factor and judged on validation data only (Table 6). The first run appeared to overfit, but stronger regularisation (run 2A) narrowed the gap between training and validation accuracy without improving results, while adding capacity (run 2B) raised macro F1 by 6.8 points despite doubling that gap. This showed that the size of the overfitting gap was a poor guide to what would improve the model. Run 2B was selected, as later runs either matched it within noise (3A) or performed worse (3B).

# 6. Results, figures and model comparison

## 6.1 Overall Performance

*Table 7: Test set performance of all four models*

| Model | Params (M) | Top-1 | Top-5 | Macro F1 | Clean test top-1 (3,166) | Top-1, 114 classes |
|---|---|---|---|---|---|---|
| Custom CNN | 4.58 | 53.84% | 84.78% | 0.532 | 53.63% | Not run |
| MobileNetV3-Large | 4.36 | 67.53% | 91.75% | 0.671 | Not computed | 68.74% |
| EfficientNetV2-S | 20.33 | 83.12% | 97.75% | 0.825 | 83.25% | 84.54% |
| Swin-T | 27.61 | 88.03% | 99.31% | 0.875 | 88.21% | 89.92% |

On the test set, Swin-T achieved the highest accuracy at 88.03 per cent, followed by EfficientNetV2-S (83.12), MobileNetV3-Large (67.53) and the custom CNN (53.84) (Table 7). For every model, macro F1 is within about one point of accuracy, so performance is spread evenly across breeds rather than concentrated in the larger classes. At almost identical size, the pretrained MobileNetV3-Large outperformed the custom CNN by 13.69 points while training for 5 epochs rather than 59. Top-5

accuracy is high for every model, from 84.78 per cent for the custom CNN to 99.31 per cent for Swin-T, so the correct breed is usually among each model's five best guesses.

## 6.2 Learning Curve

The learning curves for all models are shown in Figures 3 and 4. The pretrained models reach most of their final accuracy within the first two epochs, while the custom CNN improves gradually over 59 epochs. All models had levelled off by their final epoch.

The two shapes differ in a way that reflects where each model starts. The pretrained models are already above 70 per cent validation accuracy after a single epoch, because their features are useful before training begins and only the new classification layer has to be learned; the remaining epochs add a few points as the upper layers adapt. The custom CNN starts near chance and climbs slowly, since it has to learn edges, textures and shapes before it can learn breeds at all, and it needs 59 epochs to reach a figure the pretrained models pass in one. Its training and validation curves also separate from about epoch 20, with training loss still falling while validation loss flattens, which is the overfitting described in Section 5.5. The pretrained models show no such separation in five epochs, so their curves had not levelled off and their scores are best read as lower bounds.

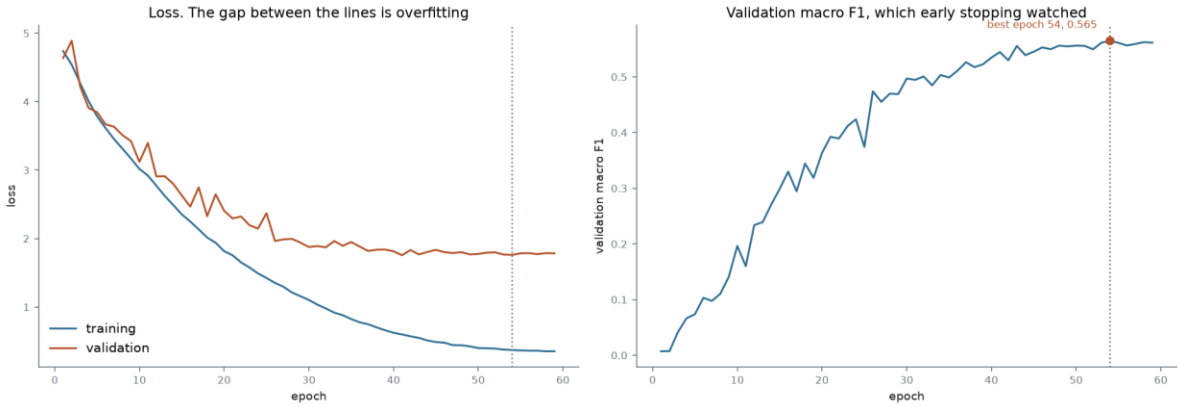



*Figure 5: Custom CNN training and validation loss, and validation macro F1*

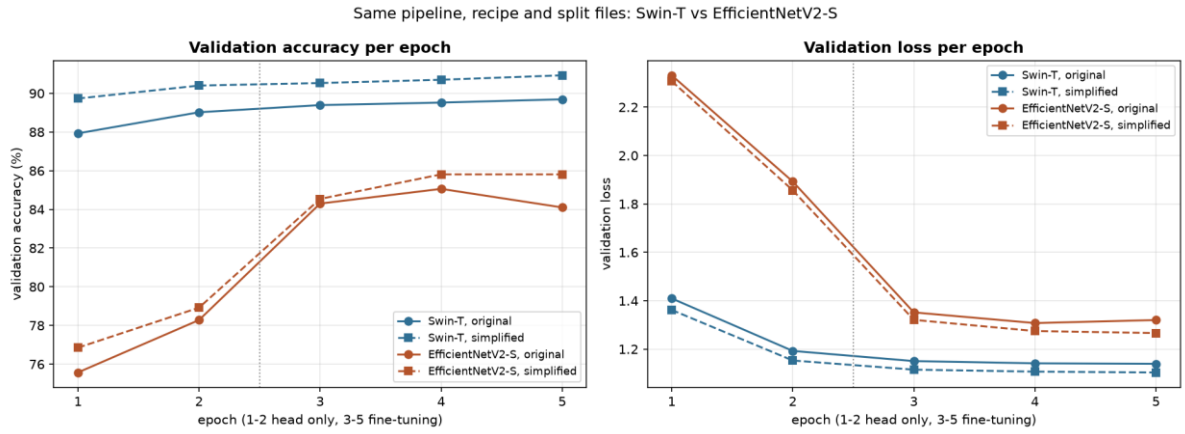



*Figure 6: Validation accuracy per epoch for the pretrained models*

## 6.3 Per-Class performance

Table 8 lists the weakest classes for the best and worst models, and full per-class precision, recall and F1 are given in Appendix A. The weakest classes are largely shared between models, with the Eskimo dog among the weakest for both.

F1 combines two things into one number between 0 and 1: precision, the share of photographs called a breed that really were that breed, and recall, the share of that breed the model actually found. A breed scoring 0.20 is therefore one the model gets wrong far more often than right, in both directions, while the strongest breeds in this project score above 0.90. Macro F1 is the plain average across all 120 breeds, so unlike accuracy it gives a rare breed the same weight as a common one, which is why it sits close to but below each model's accuracy. The weakest breeds are largely the same ones for every model, which points at the data rather than at any single architecture.

*Table 8: Weakest classes by F1 for three of the models*

| **Custom CNN** | **F1** | **MobileNetV3-Large** | **F1** | **Swin-T** | **F1** |
|---|---|---|---|---|---|
| Miniature poodle | 0.195 | Rhodesian ridgeback | 0.286 | Eskimo dog | 0.276 |
| Am. Staffordshire terrier | 0.208 | Miniature poodle | 0.294 | Collie | 0.303 |
| Eskimo dog | 0.208 | Collie | 0.316 | Lhasa | 0.480 |
| Standard schnauzer | 0.227 | Lhasa | 0.392 | | |
| Miniature schnauzer | 0.233 | Redbone | 0.394 | | |

## 6.4 Confusion matrices

Figure 7 shows the confusion matrix for Swin-T, with a zoomed view of the breeds most often confused. For the custom CNN, the Eskimo dog was predicted as a Siberian husky 10 times, more often than it was classified correctly.

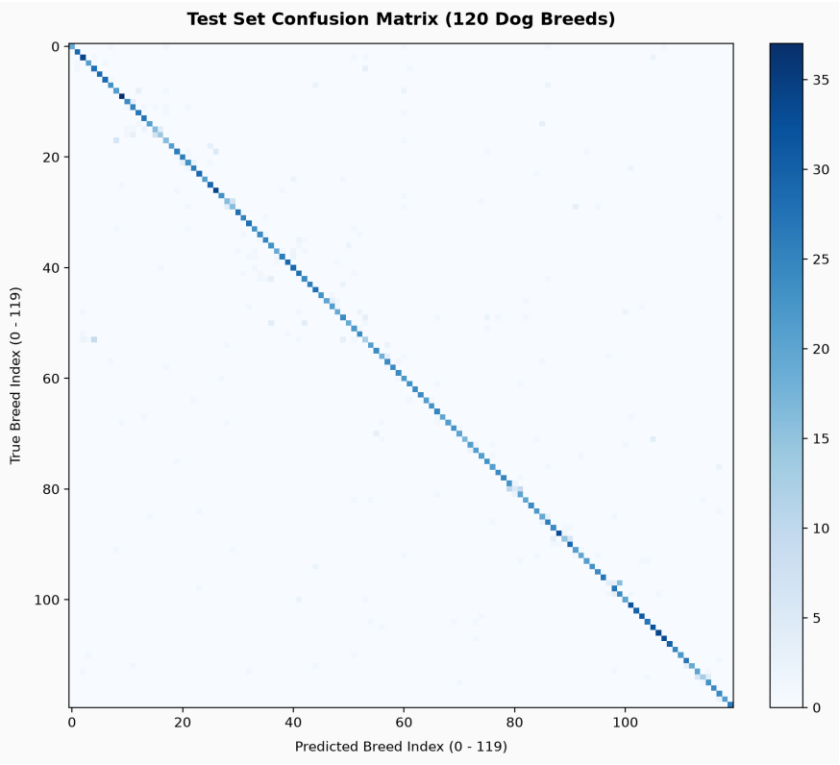
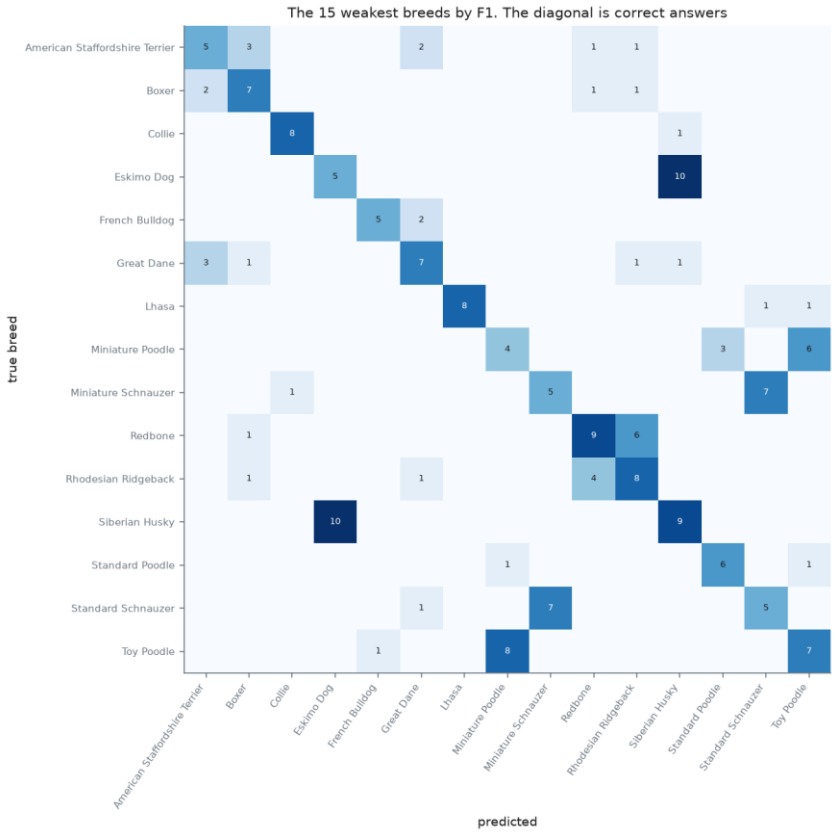



*Figure 7: Swin-T confusion matrix, with the most confused breeds enlarged*



## 6.5 Example predictions

Figure 8 shows examples of correct and incorrect predictions from Swin-T, with the predicted breed and confidence for each. The errors fall into three types, matching the dataset issues in Section 3: look-alike breeds, small dogs and photographs with more than one dog. Removing the 34 test photographs with a duplicate in training changed top-1 accuracy by less than a quarter of a point for every model, in both directions (Table 7). Merging the look-alike breeds raised accuracy for every pretrained model by 1.2 to 1.9 points, although these figures are not directly comparable with the 120-class results (Section 3.7). Only Swin-T's gain (1.89 points) clearly exceeds the noise level, which suggests that the look-alike breeds account for a small share of the remaining errors.

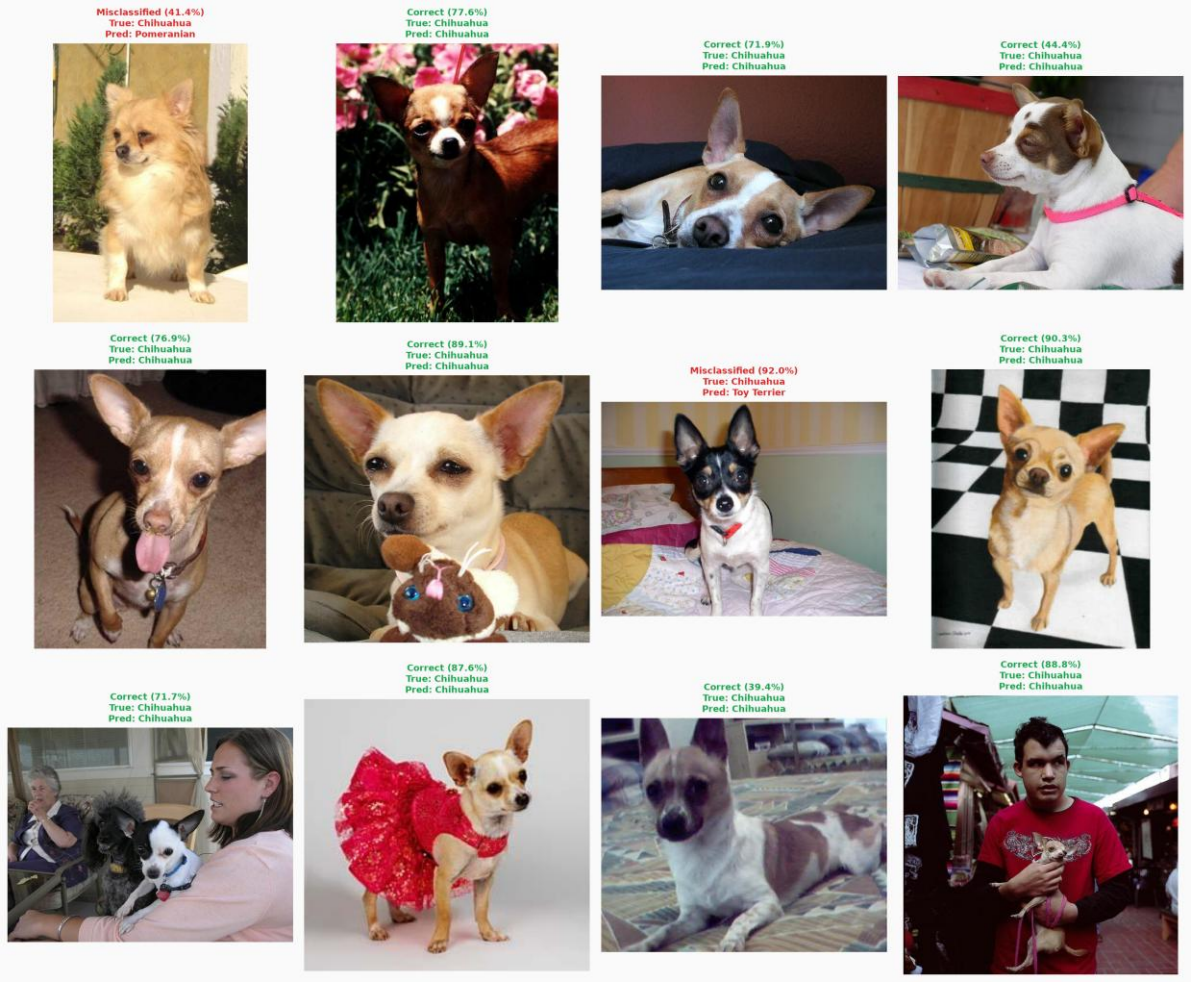



*Figure 8: Swin-T example predictions*

# 7. Discussion, limitations and error analysis

## 7.1 Why the models differ

Pretraining made the largest single difference: all three pretrained models outperformed the custom CNN, including the one of the same size. A pretrained network starts with general visual features already learned, so a small amount of training data can be spent on breed differences alone. Swin-T outperformed EfficientNetV2-S by 4.91 points, possibly because attention can relate distant parts of

the dog, such as the ears and tail, but it is also the larger model, so the two effects cannot be separated here.

## 7.2 Error analysis

The gap between the custom CNN's top-1 (53.84 per cent) and top-5 accuracy (84.78 per cent) shows that it usually narrows a photograph down to the right few breeds but then picks the wrong one. Only 7.7 per cent of its errors were made with 90 per cent confidence or more, so the model is usually uncertain when it is wrong. Its failures are therefore mostly about telling similar breeds apart, not about recognising dogs at all.

Many of the most frequent errors fall on breed pairs with contradictory labels, such as the Eskimo dog and Siberian husky. Errors that run both ways suggest that the boundary between the breeds is unclear in the data itself, rather than that the model is biased towards one breed. Swin-T's weakest classes include the same Eskimo dog and husky pair, so two very different architectures struggle on the same breeds, which points to the data rather than the models.

Some breeds can barely be separated from a photograph: the poodle and schnauzer varieties differ mainly in height, and a 224×224 image gives no sense of scale. These classes are among the weakest for the custom CNN, and they are also the groups merged in the simplified condition. Photographs with more than one dog (6.60 per cent) produce errors that standard metrics count as wrong even when the predicted breed is present, as in a schipperke photograph classified as a collie (Figure 8).

## 7.3 What the comparison supports

One way to read the 34.19-point gap is in two steps: pretraining at the same size added 13.69 points (custom CNN to MobileNetV3-Large), and moving to larger, newer architectures added a further 20.50 (MobileNetV3-Large to Swin-T). This should not be read as transfer learning being worth a fixed share of the gap, because the benefit of pretraining is likely to grow with model size, and a transformer trained from scratch on this little data would probably perform far worse than the custom CNN. MobileNetV3-Large also differs from the custom CNN in architecture, so the 13.69 points reflect pretraining and a more refined design together. Even so, MobileNetV3-Large reused settings designed for EfficientNetV2-S and trained for only 5 epochs, so its result is unlikely to overstate the benefit of pretraining.

Since Stanford Dogs comes from ImageNet, the pretrained models may have encountered some test photographs before (Section 3.1). The size of this effect was not measured, so the pretrained scores should be treated as possibly optimistic. Transfer learning remains standard and useful practice, and the point is only that on this dataset its scores need to be read with care.

## 7.4 Limitations

- Each model was trained once, so differences smaller than about 1.2 points may be noise.
- The pretrained models trained for only 5 epochs and may still have been improving.
- Possible overlap between the ImageNet pretraining data, and the test set was not measured.

- The split did not keep duplicates together, so leakage was measured rather than prevented.
- Contradictory labels and photographs with several dogs limit the accuracy any model can reach.
- Photographs were downscaled before publication and distorted by square resizing.
- Models ran on different GPUs, so training times are not directly comparable.

# 8. Conclusion

## Findings

This project compared a custom CNN, EfficientNetV2-S and Swin-T, with MobileNetV3-Large as a size-matched control, on the 120-class Stanford Dogs task. The pretrained models clearly outperformed the custom CNN, with Swin-T reaching 88.03 per cent top-1 accuracy against 53.84 per cent, and pretraining at the same model size added 13.69 points. The dataset audit found that leakage made almost no difference, but that inconsistent labels limit how accurate any model can be.

Swin-T is recommended, as it achieved the highest accuracy and macro F1 of the models tested. EfficientNetV2-S is a reasonable alternative where a convolutional model is preferred, at a cost of about 5 points. Future work should evaluate the models on a separately collected test set to remove any overlap with ImageNet. Training Swin-T from scratch would show directly how much pretraining matters for a transformer.

# 9. References

DENG, J., DONG, W., SOCHER, R., LI, L.-J., LI, K. & FEI-FEI, L. 2009. ImageNet: a large-scale hierarchical image database. Paper presented at the IEEE Conference on Computer Vision and Pattern Recognition, Miami, FL.

DOSOVITSKIY, A., BEYER, L., KOLESNIKOV, A., WEISSENBORN, D., ZHAI, X., UNTERTHINER, T., DEHGHANI, M., MINDERER, M., HEIGOLD, G., GELLY, S., USZKOREIT, J. & HOULSBY, N. 2021. An image is worth 16x16 words: transformers for image recognition at scale. Paper presented at the International Conference on Learning Representations.

HE, K., ZHANG, X., REN, S. & SUN, J. 2015. Delving deep into rectifiers: surpassing human-level performance on ImageNet classification. Paper presented at the IEEE International Conference on Computer Vision, Santiago.

HOWARD, A., SANDLER, M., CHU, G., CHEN, L.-C., CHEN, B., TAN, M., WANG, W., ZHU, Y., PANG, R., VASUDEVAN, V., LE, Q.V. & ADAM, H. 2019. Searching for MobileNetV3. Paper presented at the IEEE International Conference on Computer Vision, Seoul.

IOFFE, S. & SZEGEDY, C. 2015. Batch normalization: accelerating deep network training by reducing internal covariate shift. Paper presented at the International Conference on Machine Learning, Lille.

KHOSLA, A., JAYADEVAPRAKASH, N., YAO, B. & FEI-FEI, L. 2011. Novel dataset for fine-grained image categorization. Paper presented at the First Workshop on Fine-Grained Visual Categorization, IEEE Conference on Computer Vision and Pattern Recognition, Colorado Springs, CO.

LIU, Z., LIN, Y., CAO, Y., HU, H., WEI, Y., ZHANG, Z., LIN, S. & GUO, B. 2021. Swin Transformer: hierarchical vision transformer using shifted windows. Paper presented at the IEEE International Conference on Computer Vision, Montreal.

LOSHCHILOV, I. & HUTTER, F. 2019. Decoupled weight decay regularization. Paper presented at the International Conference on Learning Representations, New Orleans, LA.

PASZKE, A., GROSS, S., MASSA, F., LERER, A., BRADBURY, J., CHANAN, G., KILLEEN, T., LIN, Z., GIMELSHEIN, N., ANTIGA, L., DESMAISON, A., KOPF, A., YANG, E., DEVITO, Z., RAISON, M., TEJANI, A., CHILAMKURTHY, S., STEINER, B., FANG, L., BAI, J. & CHINTALA, S. 2019. PyTorch: an imperative style, high-performance deep learning library. Paper presented at Advances in Neural Information Processing Systems, Vancouver.

PYTORCH. 2026. torchvision models documentation. [Online.] Available: https://pytorch.org/vision/stable/models.html [Accessed: 18 October 2026.]

SIMONYAN, K. & ZISSERMAN, A. 2015. Very deep convolutional networks for large-scale image recognition. Paper presented at the International Conference on Learning Representations, San Diego, CA.

TAN, M. & LE, Q.V. 2021. EfficientNetV2: smaller models and faster training. Paper presented at the International Conference on Machine Learning.

# Appendix A: Per-class results

Precision, recall and F1 for all 120 breeds, for every model, are provided as CSV files in the project repository rather than reproduced here, since the full tables run to several pages. For the custom CNN see Lindani_Model/results/runs/iter2_deeper/eval_test/per_class_metrics.csv, for MobileNetV3-Large see Lindani_Model/mobilenet/results/runs/mobilenet_unclean/eval_test/per_class_metrics.csv, and for the other two models the equivalent files in JP_Model and Sulaiman_Model. Appendices do not count towards the page limit.